I am working with B-G431B-ESC1 i want to use it like a ESC for a autonomous RC car, with input in the range of [-1 to 1] and the output the estimation of the speed in the BLDC motor axel, for I/O in need to use the UART for comms with a RPi5.

So far i manage to make the identification of the motor 2852-3100 KV with a 3s Lipo 2200 mah:
- Pole pair: 2
- Rs: 0.1 omh
- Ls: 0.001 mH
- Imax: 2 Apk
- Ke: 3.0 Vrms/kRPM
- Inertia: 0.09 uN.m.s^2
- Friction 0.45 uN.m.s

Right now i had a minimun FOC running that follows the reference but does not reject disturbances.

I now i had to make more ajustments with the Motor Pilot, and the usb ST-Link can work for that, i need to mantain that connection for testing but after finish that part i want the usb to use full UART for comms with the RPi5.

Can you make a roadmap for the solution of this problem?

The attached documents are the study that so far i have been doing

I'm going to ground the roadmap in your uploaded study files first, then cross-check key B-G431B-ESC1 / MCSDK details from official ST sources so the plan matches the board and tooling you're actually using.

I checked for attached study files, but none are actually available in the chat right now, so I'm basing this on ST's board/user documentation and MCSDK references. I'm now pulling the interface details so the roadmap separates "keep ST-Link for tuning" from "final UART link to the RPi5" cleanly.

Thought for 2m 36s

Yes. For your target, I would structure it as a two-layer system:

Layer 1: ST MCSDK motor-control core on the B-G431B-ESC1
Keep the ST FOC stack as the low-level actuator layer.

Layer 2: your ESC application layer
Add a small state machine that converts a normalized command in `[-1, 1]` into safe forward, brake, neutral, and reverse behavior, and publishes estimated shaft speed back to the RPi5 over UART.

That direction fits the board well: the B-G431B-ESC1 is already an ESC-class board, it supports sensorless FOC and 6-step, exposes USB for programming/debug, and exposes J3 for PWM/UART/BEC. Its main-board pinout maps `USART2_TX` to `PB3` and `USART2_RX` to `PB4`. STMicroelectron... +1

Recommended roadmap

1) Freeze the hardware interface first

Use **USART2 on J3** as your final RPi5 communication link. On this board, J3 is the relevant I/O connector for UART/PWM/5V/GND, and the MCU pinout shows `PB3/PB4` as `USART2_TX/RX`. STMicroelectron...

Practical meaning:

- **Development mode:** keep the daughterboard + USB/ST-LINK connected so you can flash, debug, and use Motor Pilot.
- **Deployment mode:** use **J3 UART** to the RPi5 as the application interface.

I would treat **Motor Pilot mode** and **RPi5 runtime UART mode** as separate operating modes unless you bench-prove they can coexist cleanly on your exact wiring. Motor Pilot talks to the firmware through the serial UI layer using **MCPV2** over a serial port, so sharing that path with your own runtime protocol is usually the first place complexity appears. STMicroelectron... +1

2) Do not start from speed mode; start from torque-style ESC behavior

For an RC car drivetrain, the higher-level command should behave like **throttle/brake/reverse**, not "hold a target rpm" from the outside. MCSDK supports **speed mode or torque mode**, **torque ramps**, and even runtime switching between modes. STMicroelectron... +1

So your normalized input should map like this:

- `|u| < deadband` → neutral
- `u > deadband` → positive torque request
- `u < -deadband` while moving forward → braking
- `u < -deadband` and speed near zero → reverse torque request

That is the right place to implement "RC ESC feel".

3) First stabilize the existing FOC before adding car logic

You said the current FOC follows the reference but does not reject disturbances well. Before adding the RPi protocol, use Motor Pilot to work on the parts ST already exposes for runtime tuning:

- speed PID / torque PID
- observer parameters
- rev-up / startup behavior
- plotting and logging of internal registers in real time. STMicroelectron... +1

Your immediate tuning goal is not "final autonomous car behavior". It is:

1. repeatable startup
2. stable current/torque response
3. acceptable speed-estimate quality
4. no unexpected faults under small load changes

Only after that should you add the ESC state machine.

4) Build the firmware as two separable features

I would split the firmware into:

- **Motor-control core**: generated/maintained from MCSDK
- **Application layer**: your custom logic
- **Comms backend**: either Motor Pilot/MCPV2 for tuning, or custom UART protocol for RPi5 deployment

A build-time switch is a good idea here. Conceptually:

- `USE_MOTOR_PILOT = 1` for tuning
- `USE_RPI_UART = 1` for deployment

That way you do not break a known-good MCSDK project every time you touch the vehicle logic.

5) Implement a small ESC state machine

Before writing UART parsing, define the control states:

- `BOOT`
- `WAIT_NEUTRAL`
- `READY`
- `FORWARD`
- `BRAKE`
- `REVERSE`
- `FAULT`

Behavior:

- On boot, require neutral before arming.
- Allow sign change into reverse only when estimated speed is below a small threshold.
- If command reverses sign while speed is still significant, interpret it as **brake**, not immediate reverse.
- Apply a **slew/ramp** on requested torque/current.

This is where you prevent gearbox shock and traction spikes.

6) Use the board's speed estimate as telemetry first

The ST stack already supports a real-time monitor of rotor speed, and Motor Pilot is built around reading those motor-control variables over serial. STMicroelectron... +1

For your first deployment, transmit a compact telemetry packet such as:

- timestamp
- normalized command
- state
- estimated motor speed
- bus voltage
- current / torque-related variable
- fault flags

Keep this simple. Your first target is **stable and trustworthy speed telemetry**, not a rich protocol.

7) Keep the RPi5 protocol custom and minimal

Motor Pilot uses MCPV2 over serial. That is useful for ST tools, but for your autonomous car I would not make the RPi5 speak full Motor Pilot protocol unless you have a strong reason. STMicroelectron... +1

For the car, use a tiny custom protocol such as:

- command packet from RPi5: `u_cmd`, watchdog counter, enable bit
- telemetry packet from ESC: `speed_est`, state, faults, voltage

That gives you a clean separation:

- ST tools for tuning
- your own UART protocol for runtime

8) Plan for the hardest problem now: low-speed sensorless operation

For an autonomous RC car, the hardest region is usually:

- launch from stop
- crawl speed
- forward/reverse transitions under load

That is exactly where sensorless control is often least comfortable. The good news is the board supports **Hall or encoder inputs on J8**, and ST says Workbench can be configured to use those sensors during FOC.

STMicroelectron... +1

So I would define this now as a decision gate:

- **Path A:** sensorless-only, if startup + low-speed reverse are good enough on the real drivetrain
- **Path B:** Hall/encoder-assisted, if low-speed robustness is not acceptable

That avoids wasting weeks trying to force sensorless behavior into a region where the application really wants position feedback.

9) Bench sequence before installing on the car

Run in this order:

1. motor free-spin on bench
2. motor with imposed hand disturbance
3. drivetrain connected but wheels off ground
4. wheels on ground, low throttle only
5. forward/brake/reverse transitions
6. watchdog / comm-loss / fault behavior

Do not jump straight from no-load bench to full vehicle testing.

10) Define clear acceptance criteria

I would use these checkpoints:

A. Tuning checkpoint

- motor starts reliably
- no nuisance faults
- speed estimate is smooth enough for control/logging
- moderate load changes do not destabilize control

B. ESC behavior checkpoint

- neutral arming works every boot
- reverse is blocked until near-zero speed
- braking feels controlled
- torque ramps limit jerks

C. Vehicle integration checkpoint

- RPi5 command loss causes safe neutral/brake action
- telemetry is stable at your chosen update rate
- no UART-related interference with control loop

My recommended order of implementation

1. **Keep ST-Link + Motor Pilot alive**
2. **Tune the existing FOC on the bench**
3. **Switch the application to torque-style control**
4. **Add the ESC state machine**
5. **Add custom UART on USART2/J3**
6. **Send speed estimate + faults to the RPi5**
7. **Test on the real drivetrain**
8. **Only then decide whether Hall sensors are needed**

The most important design choice

If this were my project, I would choose:

- **FOC closed loop**
- **torque-commanded application layer**
- **USART2/J3 for final RPi5 comms**
- **Motor Pilot kept only as a tuning mode**
- **Hall sensors as the backup plan for low-speed robustness**

That is the shortest path from “it spins in FOC” to “it behaves like an automotive ESC”.

I do not actually see the attached study documents in this chat, so this roadmap is grounded in ST’s board and MCSDK documentation rather than your files. If you upload them here, I can turn this into a much more concrete roadmap tied to your exact project structure, including a proposed firmware architecture, task breakdown, and first implementation steps.

Sources

motor_test1_20260227_gui_v4.zip
Zip Archive

This is the project so far

Now that the project files are in the chat, I’m going to anchor the roadmap to your actual materials instead of only the generic ST path. I’ll check which parts confirm your current motor parameters, firmware state, and the UART/Motor Pilot plan, then tighten the roadmap around that.

Your uploaded material helps a lot: it already points toward the same architecture we were converging to, especially “baseline ST FOC first, then UART control + telemetry, then vehicle-style logic.” I’m pulling the exact parts that matter most for the next phase: torque-vs-speed choice, startup/handover, and how to isolate toolchain issues from control issues.

Thought for 3m 58s

I reviewed the project material, and the overall direction is good. Your own notes already outline the right sequence: first keep the ST baseline working, then understand the control pipeline, then stabilize sensorless startup and handover, and only after that add UART control and telemetry; CAN comes later.

Autonomous Driving Course - FOC...

The most important refinement is this: for an **RC-car ESC**, your runtime input `[-1, 1]` should not be treated as an external speed reference. Your files already point to the better choice: keep FOC current loops active, but use **torque / Iq control** as the driving interface, and use the estimated speed only as telemetry and for safety logic. The outline explicitly recommends torque mode for "ESC-like" behavior and reading back `SPEED_MEAS` / observer speed over UART. Autonomous Driving Course - FOC...

So I would split your project into two operating modes:

1. Tuning mode

- ST-Link + Motor Pilot active
- speed mode allowed
- focus on profiling, rev-up, observer lock, and reliable closed-loop startup

2. Deployment mode

- RPi5 talks to the board over UART
- input is normalized `u in [-1, 1]`
- application maps `u` into neutral / forward / brake / reverse
- motor-control core runs torque-mode underneath
- board streams estimated speed, current, bus voltage, state, and faults back to the Pi

That separation also matches the way your notes recommend keeping toolchain issues, profiling issues, and control issues distinct instead of mixing them together. Autonomous Driving Course - STM...

Here is the roadmap I would use from this point.

Phase 1: freeze a known-good bench baseline

Before touching the RPi protocol, keep one image whose only job is:

- start reliably
- enter closed loop reliably
- survive repeated start/stop tests
- report sensible estimated speed

Your own outline defines this well: use ST Workbench + Motor Pilot, verify Start/Stop, target speed command, and stable estimated speed in closed loop. It also recommends conservative early limits like 3–5 A phase current, 5k–10k rpm initial operating range, and 1–3 s soft ramps. Autonomous Driving Course - FOC...

Autonomous Driving Course - FOC...

Phase 2: finish sensorless startup quality

Your material is very clear that the hardest part is not UART; it is the sensorless chain:

- alignment
- open-loop ramp
- observer lock
- handover blend
- closed-loop stability

The notes already propose exactly that sequence and say the deliverable is a documented startup and handover logic with clear thresholds. They also recommend a startup success target above 90% at no-load, then light load. Autonomous Driving Course - FOC... Autonomous Driving Course - FOC...

For your car project, this phase matters even more because reverse/braking will stress the low-speed region, where sensorless estimation is weakest.

Phase 3: move the vehicle behavior out of "speed mode"

For the car, I would keep **speed mode only for tuning**. Runtime behavior should be:

- `|u| < deadband` -> neutral
- `u > deadband` -> positive torque
- `u < -deadband` while speed is still significant -> braking

- `u < -deadband` and speed is near zero -> reverse torque enabled

This part is your ESC application layer, not the MCSDK core. The MCSDK core should mainly provide:

- torque/current regulation
- estimated speed
- faults
- startup/handover state

The outline already supports this direction by recommending UART commands such as `SET_MODE`, `SET_IQ`, `STOP`, `GET_STATUS`, `GET_FAULTS`, with timeout and telemetry streaming. Autonomous Driving Course - FOC...

Phase 4: add a minimal UART protocol to the RPi5

Keep the first protocol extremely small. Your own notes already suggest a UART-first approach, DMA/ring-buffer reception, command timeout, and telemetry streaming. Autonomous Driving Course - FOC...

I would start with just these packets:

From RPi5 to ESC:

- enable
- normalized command `u`
- watchdog counter

From ESC to RPi5:

- state
- estimated speed
- `Iq_ref` / measured current
- bus voltage
- fault flags

At this stage, avoid making the Pi speak Motor Pilot protocol. Use Motor Pilot for tuning and your own protocol for runtime.

Phase 5: define “valid speed”

Your notes already warn that for sensorless operation the low-speed estimate may be noisy or invalid, and recommend publishing valid speed only once observer lock or a speed threshold has been reached. That should become part of your UART telemetry contract. Autonomous Driving Course - FOC...

So the ESC should send something like:

- `speed_valid = 0/1`
- `speed_rpm_est`
- `observer_rpm`
- `state`

That will save you trouble on the RPi side.

Phase 6: build a fault playbook before car testing

This is one of the best suggestions in your project notes: make the fault mapping explicit. The uploaded material already lists the useful structure:

- Speed Feedback -> wrong pole pairs, observer not locking, rev-up too aggressive, phase-order issue
- Over Current -> ramp too steep, current limit too high, ADC/shunt timing issue, mechanical load during profiling
- Over Heat -> real MOSFET heating, false temp sensing config, no airflow
- UV/OV -> wrong 3S thresholds or supply sag Autonomous Driving Course - STM...

For the RC car, that page is not optional. It should live next to the bench test procedure.

Phase 7: only then test the drivetrain

Follow this order:

1. free motor
2. free motor with hand disturbance
3. drivetrain connected, wheels off ground
4. low-throttle rolling test

5. forward-brake-reverse transitions
6. watchdog-loss and fault tests

That fits the testing philosophy already present in your notes: fault injection, startup success rate, speed regulation error, and current-limit respect. Autonomous Driving Course - FOC...

What I think your current project state really is

Based on the material you uploaded, the project is **not yet at the "RPi ESC" stage**. It is at the **"MCSDK baseline + tuning + sensorless startup refinement"** stage, which is exactly where it should be. The next big milestone is not "full autonomous RC car comms"; it is:

a clean split between tuning firmware and deployment firmware, with torque-mode ESC behavior on the deployment side.

One more caution from your own notes: the detent-count discussion is not a reliable pole-pair measurement by itself, so keep treating pole pairs and Ke conventions carefully, because wrong PP or wrong Ke easily produces fake observer problems. Autonomous Driving Course - STM...

My recommendation for the immediate next task is:

Create two firmware targets

- `PILOT` : Motor Pilot enabled, used only for tuning
- `RPI_ESC` : custom UART enabled, Motor Pilot disabled, runtime command = normalized torque request

Then keep the same motor-control core under both.

Send me `mc_tasks.c` , `main.c` , and `mcp_config.h` from the branch you want to keep, and I'll turn this roadmap into an exact file-by-file implementation plan.

Sources

before to start implementing, can you make a survey in the web, to search if someone else make something similar in another project? paper, conference docs, github repositories, blogs, youtube videos, etc?

I'm doing a broad scan first across ST docs, GitHub, hobby/robotics writeups, and video tutorials, then I'll narrow it to projects closest to your target: RC-car-style throttle/reverse behavior plus UART/serial integration.

I've got enough to answer the core question now: there are several close precedents, but I'm not seeing an exact one-to-one public project with all your ingredients together. The strongest matches are custom B-G431B-ESC1 actuator projects, ST forum threads on external controllers and serial links, and SimpleFOC-based bidirectional examples.

Thought for 2m 58s

Yes — I did a targeted web survey around **B-G431B-ESC1 + external host control + UART/serial + bidirectional/ESC-style behavior**.

My conclusion is:

I did not find a public project that matches your exact stack one-to-one

`B-G431B-ESC1 + autonomous RC car + normalized [-1,1] command from Raspberry Pi + speed telemetry from the ESC + sensorless FOC`

But I **did** find several **close precedents** that are very useful for your roadmap. The strongest signal is that your idea is realistic, but most public projects split into one of these families:

- ST MCSDK / Motor Pilot tuning projects
- custom actuator / quadruped / servo firmware on B-G431B-ESC1

- host-controlled motor drivers over UART/USB/CAN
- bidirectional SimpleFOC experiments on the same board. STMicroelectron... +3

1) The board itself already supports the architecture you want

ST describes the B-G431B-ESC1 as an ESC for a single BLDC/PMSM motor with **sensorless FOC**, **6-step control**, **speed regulation**, and **active braking**, and says it accepts commands from an external unit through **UART, CAN, or PWM**. ST's Motor Pilot also communicates with the firmware **through the serial port** using **MCPV2**. STMicroelectron... +1

A separate open hardware guide also confirms the board split that matters for you: **USB on the daughterboard for programming/debug**, and **J3 for PWM/UART/BECout**, which is very close to your "develop with ST-Link first, then use UART with the RPi5" plan. GitHub

2) Closest match: external controller driving the ESC over serial/UART

I found an ST community thread from **April 2024** where a user explicitly says the motor-control algorithm will run on a **Raspberry Pi 4** while the **B-G431B-ESC1** controls the motor, and asks about using **HUART2 through micro-USB**. That is not your exact final architecture, but it is very close in spirit: high-level control on a Pi, low-level actuation on the ESC. ST Community

I also found an ST community thread from **February 2024** where a user wants the B-G431B-ESC1 to run the **current/torque loop** while an **external controller** provides the higher-level speed loop. That is conceptually very close to your intended ESC layering. ST Community

3) Closest match in GitHub: B-G431B-ESC1 with two-way UART to a master board

The most relevant GitHub result I found is **mohammad-askari/stm32-esc**. It uses the **STM32-B-G431B-ESC1**, implements FOC for a robot actuator, supports **speed and torque control**, and explicitly states **two-way UART communication with the master board at 1 kHz**. It also reports control rates in the **7–13 kHz** range. This is not RC-car firmware, but from a systems point of view it is one of the closest public examples to your "RPi host + ESC slave + telemetry" goal. GitHub

4) Closest match in "custom ESC/servo firmware" projects

The best-known custom-firmware family I found is **pat92fr / ghent360** around the **B-G431B-ESC Brushless Servo Controller**. The public project states:

- custom firmware on the B-G431B-ESC1
- **position, velocity, and torque regulation**
- use of **USB/CAN** for setpoints and feedback
- a **Python GUI**
- use in a **quadruped** with multiple ESCs. Hackaday +1

That project is not an RC-car ESC, but it strongly supports the idea that the board can be turned into a **host-commanded intelligent actuator** rather than only a simple drone ESC. Hackaday +1

5) Bidirectional control examples do exist

I found a recent YouTube tutorial specifically titled **"Using simpleFOC for Bidirectional Motor Control | B-G431B-ESC1"**. That matters because your RC-car use case depends on controlled forward / brake / reverse behavior, not just one-direction spin. YouTube

Also, the official ST product page explicitly mentions **active braking**, which is directly relevant to your intended "brake before reverse" safety logic. STMicroelectron...

6) Academic / competition projects using this board

I did not find a paper exactly about "autonomous RC car with B-G431B-ESC1 over UART," but I did find academic/competition uses of the board:

A **2025 thesis** on a BLDC actuator states that the actuator is driven by a **B-G431B-ESC1** with integrated STM32G431. Diva Portal

A **RoboCup SSL team description (RoboIME 2024)** says their control system combines the **B-G431B-ESC1** with motors and feedback devices, and that their motherboard includes a **Raspberry Pi** plus **five ESC connectors** for propulsion integration. It also says Hall sensors and encoders are used for regulation. This is

not an RC car, but it is a serious robotics example showing the board in a host-supervised mobile platform.

ssl.robocup.org

7) Important pattern across the survey

The pattern I see is:

- **Exact RC-car examples are scarce**
- **Host-controlled actuator examples are real**
- **UART / USB / CAN setpoint-feedback schemes are real**
- **Bidirectional operation is being explored**
- **Higher-performance robotics projects often add sensors** rather than rely only on pure sensorless control, especially when they care about low-speed behavior and reversals. GitHub +3

That last point is especially important for you. Your target behavior — neutral, forward, brake, reverse, plus useful speed estimation at low speed — is exactly where **sensorless** systems are usually under the most stress. The public projects that care most about precise low-speed control often use **encoders or Hall sensors**. Hackaday +2

What I would take from this survey for your project

The closest practical precedent is not “someone already built your exact RC car.” It is this combination:

- use the **official ST stack** to get a stable motor-control baseline and tuning workflow
- keep the board as a **smart slave actuator**
- send commands from the host over **UART**
- return estimated speed / status / faults as telemetry
- implement RC-car behavior in your application layer
- keep open the option of adding **Hall/encoder sensing** if sensorless reverse/low-speed behavior is not good enough. STMicroelectron... +3

So the answer is: **yes, similar things have been done**, but mostly in **robot actuators, quadrupeds, servo controllers, and host-driven motor nodes**, not as a publicly documented “B-G431B-ESC1 autonomous RC car ESC with RPi UART” package. GitHub +2

Next, I can turn this survey into a **ranked shortlist of the 5 most useful repos/videos/docs for your exact implementation path**, with a note on what each one is worth copying and what to ignore.

Sources

I think i want to start with the roadmap that your make before. Just one question, the las plan has 7 phases and the previous one has 10 why the difference? i notice in state machine idea is not presented in the revised roadmap, why?

Thought for 5s

Good catch. The difference is **structure, not direction**.

The **10-step version** was a **system-level roadmap**.
It was written before I had your project files, so I separated the work into more granular blocks:

1. freeze hardware interface
2. choose torque-style behavior
3. stabilize FOC
4. build firmware layers
5. implement state machine
6. expose telemetry
7. define UART protocol
8. handle low-speed sensorless limits
9. bench test sequence
10. acceptance criteria

The **7-phase version** was a **project-execution roadmap**.
After seeing your material, I compressed related items into broader phases because your notes already

grouped several topics together. So some steps were not removed; they were **absorbed inside a larger phase**.

For example:

- In the 10-step roadmap, **state machine** was its own step.
- In the 7-phase roadmap, it was implicitly inside:
  - **Phase 3:** move vehicle behavior out of speed mode
  - **Phase 4:** add minimal UART protocol

So you are right: in the revised roadmap I did **not name it explicitly**, and that was my mistake. It should have stayed visible because for your RC-car use case it is one of the most important pieces.

Why the state machine matters so much

Because your project is not just "spin motor from UART."
It is an **ESC-like drivetrain controller**, so you need behavior rules, not only control loops.

Without a state machine, you risk:

- reverse being applied while still moving forward
- ugly transitions around neutral
- unsafe startup
- no clean reaction to UART timeout or faults

So the state machine is not optional. It is part of the application layer.

Where it should sit

It should sit **above** the motor-control core.

Think of the stack like this:

RPi5 command
-> ESC state machine / safety logic
-> torque request / current request
-> FOC current loops
-> motor

And telemetry goes upward:

observer speed / current / voltage / faults
-> state machine / app layer
-> UART to RPi5

A better merged roadmap

Here is the corrected version, combining both roadmaps cleanly:

Phase 1 — Stable ST baseline

- keep Motor Pilot working
- verify startup, handover, closed-loop behavior
- confirm estimated speed is usable

Phase 2 — Sensorless robustness

- improve rev-up
- improve observer lock
- reduce startup failures
- test light disturbances and light load

Phase 3 — Firmware architecture split

- separate:
  - motor-control core
  - application layer
  - communication layer
- create two builds:
  - tuning build

- RPi deployment build

Phase 4 — ESC state machine

Implement explicitly:

- `BOOT`
- `WAIT_NEUTRAL`
- `READY`
- `FORWARD`
- `BRAKE`
- `REVERSE`
- `FAULT`

With rules such as:

- neutral required at startup
- sign reversal first becomes brake
- reverse enabled only near zero speed
- timeout forces safe state
- faults force safe stop

Phase 5 — UART protocol with the RPi5

- normalized command `u in [-1,1]`
- enable bit / watchdog
- telemetry: estimated speed, state, voltage, current, faults

Phase 6 — Bench and drivetrain testing

- free motor
- disturbance by hand
- drivetrain off-ground
- low-speed rolling
- forward/brake/reverse transitions

Phase 7 — Acceptance criteria and iteration

- startup success rate
- speed estimate quality
- safe reversals
- timeout behavior
- real-car usability

So why did it disappear?

Not because it stopped being important.

It disappeared because I **compressed the roadmap too aggressively** after reading your notes. I grouped “vehicle behavior” into a broader phase, but for your case the state machine deserves its own heading.

For your project, I would keep the **7-phase roadmap**, but with **Phase 4 explicitly dedicated to the state machine**.

That would be the cleanest version going forward.

The next thing I would do is define that state machine in detail before any UART implementation.

if i ask, what would be the bare minimum for this project, what would you response?

Thought for a few seconds

The bare minimum would be:

Make the B-G431B-ESC1 behave as a safe UART-controlled motor node, not yet a “full RC-car ESC.”

In practical terms, that means only these essentials:

1. A stable motor-control baseline

The motor must:

- start reliably
- spin in both directions, even if reverse is still conservative
- report a usable estimated speed
- not fault constantly

Without this, nothing above it matters.

2. One simple UART command from the RPi5

Only one input is necessary at first:

- `u` in `[-1, 1]`

and you map it very simply:

- near 0 -> stop
- positive -> forward torque
- negative -> reverse torque

At the absolute minimum, you could even delay "smart braking" and use a very cautious sign change policy.

3. One telemetry output

Only send back:

- estimated motor speed

That alone already lets the Pi know whether the board is alive and what the motor is doing.

4. Basic safety

Minimum safety should still include:

- neutral / zero command on startup
- command timeout over UART
- fault -> stop motor

This is the minimum I would never remove.

So the true bare minimum is:

closed-loop FOC running + UART input `[-1,1]` + speed telemetry + timeout/fault stop

Not yet required for the bare minimum:

- polished state machine
- refined brake-before-reverse behavior
- advanced telemetry
- low-speed perfection
- drivetrain tuning on the real car
- Motor Pilot coexistence with runtime UART
- autonomous vehicle integration logic

If I say it in one sentence:

The bare minimum is to turn the board into a UART-controlled smart actuator that accepts a normalized command and returns estimated speed safely.

For your project, I would define the first success milestone as:

"From the RPi5, I can send a value in `[-1,1]`, the motor responds reproducibly, and the board streams back estimated speed without unsafe behavior."

That would be enough to call the first version real and usable.

If we try to implement the bare minimum first, can we improved later to get the full previous plan?

Thought for 4 seconds

Yes — that is the best way to do it.

But it only works well if the bare minimum is built as a **foundation**, not as a throwaway shortcut.

What should stay the same from day one:

- the **MCSDK motor-control core**
- the **UART interface to the RPi5**
- the idea that the RPi sends a **normalized command**
- the idea that the ESC returns **speed telemetry**
- the separation between **low-level motor control** and **high-level vehicle behavior**

What can be simple at first, then improved later:

- the mapping from `[-1,1]` to motor action
- the safety logic
- the telemetry richness
- the transition logic for brake/reverse
- the tuning quality under load
- the robustness of startup and low-speed operation

So the evolution can be very clean:

Version 1: bare minimum

- UART receives `u in [-1,1]`
- simple mapping to torque/current request
- send back estimated speed
- timeout and fault stop

Version 2: safer behavior

- neutral-on-startup
- deadband
- ramp limiting
- better fault handling

Version 3: real ESC behavior

- explicit state machine
- brake-before-reverse
- reverse only near zero speed
- better telemetry and watchdogs

Version 4: vehicle-ready

- drivetrain tuning
- load testing
- improved low-speed behavior
- maybe Hall/encoder if sensorless is not enough

So yes, you can absolutely start with the bare minimum **and grow into the full plan**.

The key is to avoid these mistakes in version 1:

- do not hard-code everything inside autogenerated MCSDK files
- do not mix UART parsing directly with control law logic
- do not assume speed mode is your final interface
- do not design a protocol that you will need to throw away

If you keep a small architecture like this from the start:

UART parser -> command variable -> application layer -> motor-control core
and
motor variables -> telemetry layer -> UART

then the full roadmap becomes an extension of the first version, not a rewrite.

So my answer is:

ChatGPT 5.4 Thinking

Share

Build the smallest working smart actuator first, then grow it into the full RC-car ESC step by step.

Okay then. can we start now with the "smart actuator"?

Yes. I'll start by locating the project structure in the uploaded zip and identifying the safest insertion points for the smart-actuator layer, so we preserve the MCSDK core and add only the minimum UART command + speed telemetry path first.

I found the right project shape: it already has the MCSDK-generated core plus the serial/MCP pieces, which is ideal for a smart-actuator first version. The key files are exactly where we want them: `main.c`, `mc_tasks.c`, `motorcontrol.c`, `mcp.c`, `mcp_config.*`, and the `.ioc`.

Thought for 2m 21s

Yes — and the first thing to do is **not** write the actuator logic yet.

The first thing is to **free USART2 from Motor Pilot when you are in smart-actuator mode**.

From your project, the key point is this:

- `main.c` initializes `USART2`
- `mcp_config.h` binds Motor Pilot/MCP to `USART2`
- `MCboot()` starts `ASPEP_start(&aspepOverUartA);`
- `MC_RunMotorControlTasks()` keeps processing MCP packets on that same UART

So right now, **USART2 already belongs to Motor Pilot**.
That means the clean bare-minimum architecture is:

Step 0 — split the project into two modes

Create two firmware modes:

- **Pilot mode**
  keep Motor Pilot over USART2 for tuning
- **Smart actuator mode**
  disable MCP/ASPEP over USART2 and use USART2 for the RPi5 protocol

This is the safest first move because it preserves your working tuning path.

Step 1 — first success target

Let us define a very small target for version 1:

- RPi sends one command `u in [-1,1]`
- board maps `u` to motor action
- board returns estimated speed
- timeout stops the motor
- faults stop the motor

That is enough for a real first milestone.

Step 2 — keep the architecture clean

I would add a new application layer instead of modifying the MCSDK core everywhere.

Use something like this:

- `smart_actuator.h`
- `smart_actuator.c`

with functions like:

- `SA_Init()`
- `SA_OnRxLine(...)`
- `SA_Run_1ms()`
- `SA_GetTelemetryLine(...)`

And keep the data inside a small struct:

- latest command `u`
- command timeout timer
- enabled flag
- motor started flag
- last telemetry values

Step 3 — choose torque path from day one

Even for the bare minimum, I would **not** build this around external speed mode.

Use:

- `u > 0` -> positive torque request
- `u < 0` -> negative torque request
- `u ~ 0` -> zero torque / stop request

Why:

- it matches ESC behavior better
- it will grow more naturally into forward/brake/reverse logic later
- it avoids a rewrite later

For the very first version, this mapping can be simple and conservative.

Step 4 — simplest protocol first

For version 1, use a very simple **ASCII line protocol** over UART.

Example from RPi:

```
Plain text
```

Example from ESC:

```
Plain text
```

Why ASCII first:

- easy to test with serial terminal
- easy to generate from Python on the RPi
- easy to debug
- easy to replace later with binary if needed

Step 5 — where to place the code

I would place it like this:

In `main.c`

Use USER CODE sections to:

- include `smart_actuator.h`
- call `SA_Init()` after `MX_MotorControl_Init()`
- call `SA_Run_1ms()` or `SA_Run()` in the main loop

In `mc_tasks.c`

Use USER CODE sections only if needed to:

- read motor state / estimated speed periodically
- keep app logic synchronized with the medium-frequency task

But for the first version, most application logic can stay outside and just use MCSDK API calls.

Step 6 — first APIs to use from MCSDK

For the smart actuator, the most useful existing APIs are already there:

- start motor
- stop motor
- execute torque ramp
- read average mechanical speed
- read faults
- read state machine state

So the first application loop can do something like:

1. parse UART command
2. refresh timeout
3. if timeout -> stop motor
4. if no fault and command nonzero:
    - start motor if needed
    - apply torque ramp
5. if command zero:
    - apply zero torque or stop motor
6. send estimated speed telemetry

Step 7 — the very first implementation order

This is the order I recommend right now:

Phase A

Add a compile flag such as:

```c
```

Phase B

Guard these two places:

- `ASPEP_start(&aspepOverUartA);`
- MCP packet processing inside `MC_RunMotorControlTasks()`

So when `USE_MOTOR_PILOT == 0`, USART2 is free for your actuator protocol.

Phase C

Create `smart_actuator.c/.h`

Phase D

Implement:

- UART receive buffer
- line parser for `CMD x.xx`
- timeout watchdog
- speed telemetry print

Phase E

Connect the actuator logic to MCSDK APIs

What we should not do yet

Not yet:

- full ESC state machine
- brake-before-reverse
- advanced binary protocol

- Motor Pilot coexistence on the same port
- polished telemetry packets

Those come later.

My recommendation for the very next concrete move

Start with this exact goal:

"Build a smart-actuator mode that disables Motor Pilot on USART2 and accepts `CMD <float>` over UART, then returns estimated speed."

That is the right first brick.

In your current project, the first code change should be the MCP gating, because that is the real blocker right now.

Excellent. Please give me a detail step-by-step guide of how i have to change and include this new feature in my project?

I'm opening the project zip and mapping the exact files/lines we need to touch, so the guide matches your repository instead of staying generic.

I confirmed the main blocker in your repo: USART2 is already fully owned by the Motor Pilot path. So the first version of the guide has to begin by cleanly "taking back" USART2 before we add any smart-actuator code.

Thought for 7m 25s

Yes. For the **first smart-actuator version**, I would implement:

- **UART in polling mode**
- **torque/Iq-style command from `[-1, 1]`**
- **speed telemetry out**
- **Motor Pilot disabled in this build**

That matches the path you had already defined: first get closed-loop FOC stable, then add UART control and telemetry, and for ESC-like behavior use **torque/Iq control** rather than external speed mode. Your earlier notes also warned that speed should only be treated as valid once the observer is trustworthy.

Autonomous Driving Course - FOC... Autonomous Driving Course - FOC... Autonomous Driving Course - FOC...

For this very first version, I am choosing **polling UART**, not DMA+ring buffer, because it is the simplest way to free USART2 from Motor Pilot and get a working actuator quickly. That is also the simplest UART programming mode in STM32 when the UART is not yet a critical real-time peripheral. Mastering STM32

Also: do the first tests **with the drivetrain unloaded / wheels off the ground** and keep the current conservative at first. That is consistent with your earlier bench procedure. Autonomous Driving Course - FOC...

What you are going to change

You will touch these files:

- `Inc/main.h`
- `Src/main.c`
- `Src/mc_tasks.c`
- `Src/stm32_mc_common_it.c`

And you will create two new files:

- `Inc/smart_actuator.h`
- `Src/smart_actuator.c`

Before doing anything else:

Step 0 — protect your current working project

Do this first.

Create a copy or a new git branch, for example:

```bash
git checkout -b smart_actuator_v1
```

Reason: some of the files we will edit are generated by ST tools, so later regeneration can overwrite them.

Also, for now:

- do **not** regenerate from Motor Control Workbench
- do **not** regenerate from CubeMX

First make this version work.

Step 1 — add a build switch in `main.h`

Open:

`Inc/main.h`

Inside the section:

```c
/* USER CODE BEGIN Private defines */
```

add this:

```c
#define USE_MOTOR_PILOT 0
#define SMART_ACTUATOR_UART_BAUD 115200
```

So that part becomes:

```c
/* USER CODE BEGIN Private defines */
#define USE_MOTOR_PILOT 0
#define SMART_ACTUATOR_UART_BAUD 115200
/* USER CODE END Private defines */
```

What this does:

- `USE_MOTOR_PILOT = 0` means this firmware build is for the RPi smart actuator
- later, if you want to go back to Motor Pilot, set it to `1`

Step 2 — change USART2 baud rate in `main.c`

Open:

`Src/main.c`

Find `MX_USART2_UART_Init()`.

Right now your project sets:

```c
huart2.Init.BaudRate = 1843200;
```

Replace it with:

```c
#if USE_MOTOR_PILOT
  huart2.Init.BaudRate = 1843200;
```

```c
#else
  huart2.Init.BaudRate = SMART_ACTUATOR_UART_BAUD;
#endif
```

Why:

- `1843200` is fine for Motor Pilot
- for the first RPi/serial tests, `115200` is much easier to work with

---

Step 3 — disable Motor Pilot ownership of USART2 in `mc_tasks.c`

Open:

`Src/mc_tasks.c`

3.1 Disable ASPEP start in `MCboot()`

Find this line inside `MCboot()`:

```c
ASPEP_start(&aspepOverUartA);
```

Replace it with:

```c
#if USE_MOTOR_PILOT
    ASPEP_start(&aspepOverUartA);
#endif
```

3.2 Disable MCP packet handling in `MC_RunMotorControlTasks()`

Find this whole block:

```c
      MCP_Over_UartA.rxBuffer = MCP_Over_UartA.pTransportLayer->fRXPacketProcess(MCP_Over_UartA.p
                                                                                 &MCP_Over_UartA.r
      if ( 0U == MCP_Over_UartA.rxBuffer)
      {
        /* Nothing to do */
      }
      else
      {
        /* Synchronous answer */
        if (0U == MCP_Over_UartA.pTransportLayer->fGetBuffer(MCP_Over_UartA.pTransportLayer,
                                                             (void **) &MCP_Over_UartA.txBuffer,
                                                             MCTL_SYNC))
        {
          /* No buffer available to build the answer ... should not occur */
        }
        else
        {
          MCP_ReceivedPacket(&MCP_Over_UartA);
          MCP_Over_UartA.pTransportLayer->fSendPacket(MCP_Over_UartA.pTransportLayer, MCP_Over_Ua
                                                      MCP_Over_UartA.txLength, MCTL_SYNC);
        }
      }
```

Wrap it like this:

```c
#if USE_MOTOR_PILOT
      MCP_Over_UartA.rxBuffer = MCP_Over_UartA.pTransportLayer->fRXPacketProcess(MCP_Over_UartA.p
                                                                                 &MCP_Over_UartA.r
      if ( 0U == MCP_Over_UartA.rxBuffer)
      {
        /* Nothing to do */
      }
      else
      {
        /* Synchronous answer */
        if (0U == MCP_Over_UartA.pTransportLayer->fGetBuffer(MCP_Over_UartA.pTransportLayer,
```

```
                                                    (void **) &MCP_Over_UartA.txBuffer,
                                                    MCTL_SYNC))
          {
            /* No buffer available to build the answer ... should not occur */
          }
          else
          {
            MCP_ReceivedPacket(&MCP_Over_UartA);
            MCP_Over_UartA.pTransportLayer->fSendPacket(MCP_Over_UartA.pTransportLayer, MCP_Over_Ua
                                                        MCP_Over_UartA.txLength, MCTL_SYNC);
          }
        }
#endif
```

This is the key step that frees USART2 logically from Motor Pilot.

Step 4 — disable the Motor Pilot USART interrupt path

Open:

`Src/stm32_mc_common_it.c`

4.1 Change `USART2_IRQHandler()`

Inside `USART2_IRQHandler()`, wrap the existing body like this:

C

```c
void USART2_IRQHandler(void)
{
#if USE_MOTOR_PILOT
  /* existing current body stays exactly here */
#else
  /* Smart actuator v1 uses polling mode, so USART2 IRQ does nothing */
#endif
}
```

Do not rewrite the Motor Pilot code. Just wrap it.

4.2 Change the SysTick DMA-RX MCP handling

In `SysTick_Handler()`, find this block:

C

```c
  if (LL_DMA_IsActiveFlag_TC(DMA_RX_A, DMACH_RX_A))
  {
    LL_DMA_ClearFlag_TC(DMA_RX_A, DMACH_RX_A);
    ASPEP_HWDataReceivedIT(&aspepOverUartA);
  }
  else
  {
    /* Nothing to do */
  }
```

Wrap it like this:

C

```c
#if USE_MOTOR_PILOT
  if (LL_DMA_IsActiveFlag_TC(DMA_RX_A, DMACH_RX_A))
  {
    LL_DMA_ClearFlag_TC(DMA_RX_A, DMACH_RX_A);
    ASPEP_HWDataReceivedIT(&aspepOverUartA);
  }
  else
  {
    /* Nothing to do */
  }
#endif
```

Now the SysTick keeps running motor control, but no longer services MCP-over-UART.

Step 5 — disable USART2 and DMA RX NVIC for this build

Open:

`Src/main.c`

Find `MX_NVIC_Init()`.

Replace the USART2 and DMA1 Channel 1 interrupt config with this:

```c
#if USE_MOTOR_PILOT
  /* USART2_IRQn interrupt configuration */
  HAL_NVIC_SetPriority(USART2_IRQn, 3, 1);
  HAL_NVIC_EnableIRQ(USART2_IRQn);

  /* DMA1_Channel1_IRQn interrupt configuration */
  HAL_NVIC_SetPriority(DMA1_Channel1_IRQn, 3, 0);
  HAL_NVIC_EnableIRQ(DMA1_Channel1_IRQn);
#endif
```

Leave the motor-control interrupts exactly as they are:

- `TIM1_BRK_TIM15_IRQn`
- `TIM1_UP_TIM16_IRQn`
- `ADC1_2_IRQn`
- `EXTI15_10_IRQn`

Do not touch those.

Step 6 — create the smart actuator header

Create:

`Inc/smart_actuator.h`

Paste this:

```c
#ifndef SMART_ACTUATOR_H
#define SMART_ACTUATOR_H

#include "main.h"

void SA_Init(UART_HandleTypeDef *huart);
void SA_Run(void);

#endif
```

Step 7 — create the smart actuator source

Create:

`Src/smart_actuator.c`

Paste this:

```c
#include "smart_actuator.h"
#include "mc_api.h"
#include "mc_interface.h"

#include <string.h>
#include <stdlib.h>
#include <stdio.h>
#include <math.h>

#define SA_RX_LINE_MAX              64U
#define SA_CMD_TIMEOUT_MS           300U
```

```c
#define SA_CTRL_PERIOD_MS           10U
#define SA_TEL_PERIOD_MS            50U

#define SA_DEADBAND                 0.05f
#define SA_IQ_MAX_A                 0.8f
#define SA_RAMP_MS                  100U
#define SA_START_RAMP_MS            300U

#define SA_SIGN_CHANGE_BLOCK_RPM    300.0f
#define SA_STOP_RPM                 150.0f
#define SA_SPEED_VALID_RPM          500.0f

static UART_HandleTypeDef *s_huart = NULL;

static char s_rx_line[SA_RX_LINE_MAX];
static uint32_t s_rx_idx = 0U;

static float s_cmd_u = 0.0f;
static uint32_t s_last_cmd_ms = 0U;
static uint32_t s_last_ctrl_ms = 0U;
static uint32_t s_last_tel_ms = 0U;

static float sa_clampf(float x, float xmin, float xmax)
{
  if (x < xmin) return xmin;
  if (x > xmax) return xmax;
  return x;
}

static void SA_SendText(const char *txt)
{
  if ((s_huart != NULL) && (txt != NULL))
  {
    HAL_UART_Transmit(s_huart, (uint8_t *)txt, (uint16_t)strlen(txt), 20U);
  }
}

static void SA_ProcessLine(const char *line)
{
  if (line == NULL) return;

  if (strncmp(line, "CMD ", 4) == 0)
  {
    float u = strtof(&line[4], NULL);
    s_cmd_u = sa_clampf(u, -1.0f, 1.0f);
    s_last_cmd_ms = HAL_GetTick();
    SA_SendText("OK\r\n");
    return;
  }

  if (strcmp(line, "STOP") == 0)
  {
    s_cmd_u = 0.0f;
    s_last_cmd_ms = HAL_GetTick();
    MC_ProgramTorqueRampMotor1_F(0.0f, SA_RAMP_MS);
    (void)MC_StopMotor1();
    SA_SendText("OK\r\n");
    return;
  }

  if (strcmp(line, "PING") == 0)
  {
    SA_SendText("PONG\r\n");
    return;
  }

  SA_SendText("ERR\r\n");
}

static void SA_PollRx(void)
{
  uint8_t ch;

  if (s_huart == NULL) return;

  while (HAL_UART_Receive(s_huart, &ch, 1U, 0U) == HAL_OK)
  {
    if ((ch == '\r') || (ch == '\n'))
    {
      if (s_rx_idx > 0U)
      {
```

```
        s_rx_line[s_rx_idx] = '\0';
        SA_ProcessLine(s_rx_line);
        s_rx_idx = 0U;
      }
    }
    else
    {
      if (s_rx_idx < (SA_RX_LINE_MAX - 1U))
      {
        s_rx_line[s_rx_idx++] = (char)ch;
      }
      else
      {
        s_rx_idx = 0U;
        SA_SendText("ERR\r\n");
      }
    }
  }
}

static void SA_ControlStep(void)
{
  uint32_t now = HAL_GetTick();
  float speed_rpm;
  float u;
  float iq_ref_a;
  uint16_t faults;
  MCI_State_t st;

  if ((now - s_last_ctrl_ms) < SA_CTRL_PERIOD_MS)
  {
    return;
  }
  s_last_ctrl_ms = now;

  speed_rpm = MC_GetAverageMecSpeedMotor1_F();
  faults = MC_GetCurrentFaultsMotor1();
  st = MC_GetSTMStateMotor1();

  u = s_cmd_u;
  if ((now - s_last_cmd_ms) > SA_CMD_TIMEOUT_MS)
  {
    u = 0.0f;
  }

  if ((faults != MC_NO_FAULTS) || (st == FAULT_NOW) || (st == FAULT_OVER))
  {
    MC_ProgramTorqueRampMotor1_F(0.0f, SA_RAMP_MS);
    (void)MC_StopMotor1();
    return;
  }

  if (fabsf(u) <= SA_DEADBAND)
  {
    MC_ProgramTorqueRampMotor1_F(0.0f, SA_RAMP_MS);

    if (fabsf(speed_rpm) < SA_STOP_RPM)
    {
      (void)MC_StopMotor1();
    }
    return;
  }

  /* crude safety rule for v1:
     if command sign and measured speed sign disagree strongly,
     force zero torque instead of instant reversal */
  if (((u > 0.0f) && (speed_rpm < -SA_SIGN_CHANGE_BLOCK_RPM)) ||
      ((u < 0.0f) && (speed_rpm >  SA_SIGN_CHANGE_BLOCK_RPM)))
  {
    MC_ProgramTorqueRampMotor1_F(0.0f, SA_RAMP_MS);
    return;
  }

  iq_ref_a = u * SA_IQ_MAX_A;

  if (st == IDLE)
  {
    MC_ProgramTorqueRampMotor1_F(iq_ref_a, SA_START_RAMP_MS);
    (void)MC_StartMotor1();
  }
  else
```

```
  {
    MC_ProgramTorqueRampMotor1_F(iq_ref_a, SA_RAMP_MS);
  }
}

static void SA_SendTelemetry(void)
{
  uint32_t now = HAL_GetTick();
  float speed_rpm_f;
  int speed_rpm_i;
  int u_milli;
  int speed_valid;
  uint16_t faults;
  MCI_State_t st;
  char msg[96];

  if ((now - s_last_tel_ms) < SA_TEL_PERIOD_MS)
  {
    return;
  }
  s_last_tel_ms = now;

  speed_rpm_f = MC_GetAverageMecSpeedMotor1_F();
  speed_rpm_i = (int)speed_rpm_f;
  u_milli = (int)(s_cmd_u * 1000.0f);
  speed_valid = (MC_GetSpeedSensorReliabilityMotor1() && (fabsf(speed_rpm_f) > SA_SPEED_VALID_RPM
  faults = MC_GetCurrentFaultsMotor1();
  st = MC_GetSTMStateMotor1();

  (void)snprintf(msg, sizeof(msg),
                 "TEL u=%d spd=%d valid=%d st=%d flt=%u\r\n",
                 u_milli, speed_rpm_i, speed_valid, (int)st, (unsigned int)faults);

  SA_SendText(msg);
}

void SA_Init(UART_HandleTypeDef *huart)
{
  s_huart = huart;
  s_rx_idx = 0U;
  s_cmd_u = 0.0f;
  s_last_cmd_ms = HAL_GetTick();
  s_last_ctrl_ms = HAL_GetTick();
  s_last_tel_ms = HAL_GetTick();

  SA_SendText("SA READY\r\n");
}

void SA_Run(void)
{
  SA_PollRx();
  SA_ControlStep();
  SA_SendTelemetry();
}
```

Step 8 — connect the new module in `main.c`

Open:

`Src/main.c`

8.1 Add the include

Inside:

```c
/* USER CODE BEGIN Includes */
```

add:

```c
#include "smart_actuator.h"
```

8.2 Initialize it

Inside `main()`, after:

```c
MX_MotorControl_Init();
```

add:

```c
#if !USE_MOTOR_PILOT
  SA_Init(&huart2);
#endif
```

So that part becomes:

```c
MX_USART2_UART_Init();
MX_MotorControl_Init();

#if !USE_MOTOR_PILOT
  SA_Init(&huart2);
#endif
```

8.3 Run it in the main loop

Inside the infinite loop:

```c
while (1)
{
}
```

replace with:

```c
while (1)
{
#if !USE_MOTOR_PILOT
  SA_Run();
#endif
}
```

Step 9 — build

Now build the project.

If you created the files from inside CubeIDE, they should be included automatically.

If you copied them from Windows Explorer instead:

- right click project
- Refresh
- then Build

Step 10 — first bench test procedure

For this first power-up:

- wheels off the ground
- preferably no heavy drivetrain load
- fan on the ESC if possible
- use a small command first

That is exactly the right order for adding UART after a stable FOC baseline. Autonomous Driving Course - FOC...

Expected serial behavior

At reset, you should receive:

```
SA READY
```

If you send:

```
PING
```

you should get:

```
PONG
```

If you send:

```
CMD 0.10
```

you should get periodic telemetry like:

```
TEL u=100 spd=0 valid=0 st=...
TEL u=100 spd=650 valid=1 st=...
TEL u=100 spd=1100 valid=1 st=...
```

Try this sequence:

```
CMD 0.05
CMD 0.10
CMD 0.00
CMD -0.05
STOP
```

Do not start with large commands.

Step 11 — what this first version does and does not do

It does

- accept one normalized command `CMD x`
- map it to torque request
- start the motor if needed
- send estimated speed back
- stop on timeout
- stop on faults
- block crude instant sign-change when speed is still strongly opposite

It does not yet do

- full ESC state machine
- polished brake-before-reverse
- DMA ring buffer UART
- Motor Pilot + RPi coexistence on the same UART
- rich telemetry packet format

That is intentional. This is the **bare minimum smart actuator**.

Step 12 — very important warning about regeneration

The edits in:

- `mc_tasks.c`
- `stm32_mc_common_it.c`

are in generated MCSDK files.

So for now:

- do not regenerate from Workbench
- do not regenerate from CubeMX

Later, when this works, we can move the changes into a cleaner long-term structure.

Step 13 — first tuning values for v1

For your first UART-controlled torque tests, keep these conservative:

- `SA_IQ_MAX_A = 0.8f`
- `SA_DEADBAND = 0.05f`
- `SA_CMD_TIMEOUT_MS = 300`
- `SA_TEL_PERIOD_MS = 50`
- `SA_SIGN_CHANGE_BLOCK_RPM = 300`

Why this choice:

- the board already has sensorless low-speed limitations
- torque/Iq control is the cleaner ESC-like first step
- speed should not be fully trusted until observer lock / reliability is reasonable.

Autonomous Driving Course - FOC... Autonomous Driving Course - FOC...

Minimal summary of the file edits

`Inc/main.h`

Add:

```c
#define USE_MOTOR_PILOT 0
#define SMART_ACTUATOR_UART_BAUD 115200
```

`Src/main.c`

- include `smart_actuator.h`
- make USART2 baud conditional
- call `SA_Init(&huart2);`
- call `SA_Run();`
- disable USART2/DMA NVIC when not using Motor Pilot

`Src/mc_tasks.c`

- guard `ASPEP_start(...)`
- guard MCP packet processing block

`Src/stm32_mc_common_it.c`

- guard `USART2_IRQHandler()`
- guard MCP DMA RX processing in `SysTick_Handler()`

new files

- `Inc/smart_actuator.h`
- `Src/smart_actuator.c`

If you do these steps exactly and the build gives you the first error, paste that error and I'll walk you through the next fix line by line.

Sources